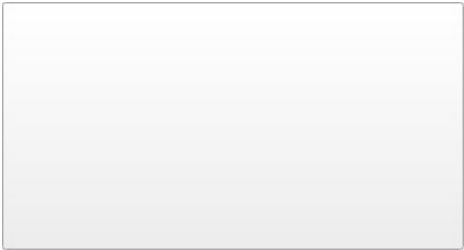


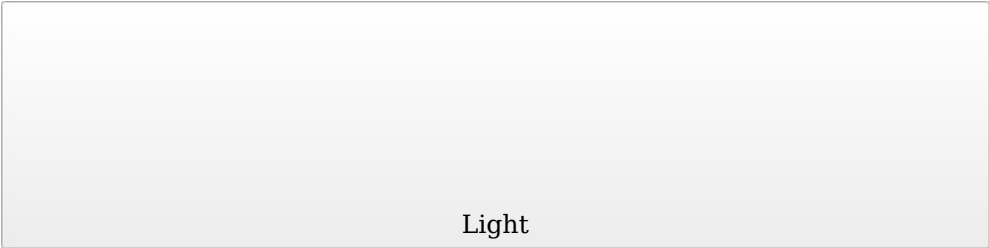
OpenTelemetry

- [Docs](#)
- [Ecosystem](#)
- [Status](#)
- [Community](#)
- [Training](#)
- [Blog](#)
- [English EN](#)
 - [日本語](#)
 - [English](#)
 - [Español](#)
 - [Français](#)
 - [日本語](#)
 - [Polski](#)
 - [Português](#)
 - [Română](#)
 - [Українська](#)
 - [中文](#)

•



◦



Light

◦



Dark

◦



Auto



- [Docs](#)
 - ☐ [What is OpenTelemetry?](#)
 - ☐ [Getting Started](#)
 - ☐ [Dev](#)
 - ☐ [Ops](#)
 - ☒ [Concepts](#)
 - ☐ [Observability primer](#)
 - ☐ [Context propagation](#)
 - ☐ [Signals](#)
 - ☐ [Traces](#)
 - ☐ [Metrics](#)
 - ☐ [Logs](#)
 - ☐ [Baggage](#)
 - ☐ [Profiles](#)
 - ☐ [Instrumentation](#)
 - ☐ [Zero-code](#)
 - ☐ [Code-based](#)
 - ☐ [Libraries](#)
 - ☐ [Components](#)
 - ☐ [Semantic Conventions](#)
 - ☐ [Resources](#)
 - ☐ [Instrumentation scope](#)
 - ☒ [Sampling](#)
 - ☐ [Distributions](#)
 - ☐ [Glossary](#)
 - ☐ [Demo](#)
 - ☐ [Architecture](#)
 - ☐ [Collector Data Flow Dashboard](#)
 - ☐ [Development](#)
 - ☐ [Docker](#)
 - ☐ [Feature Flags](#)
 - ☐ [Diagnosing memory leaks](#)
 - ☐ [Forking](#)
 - ☐ [Kubernetes](#)
 - ☐ [Requirements](#)
 - ☐ [Application](#)
 - ☐ [Architecture](#)
 - ☐ [OTel Requirements](#)
 - ☐ [System](#)
 - ☐ [Sample Configurations](#)
 - ☐ [Tail Sampling](#)
 - ☐ [Screenshots](#)
 - ☐ [Services](#)
 - ☐ [Accounting](#)
 - ☐ [Ad](#)
 - ☐ [Cart](#)
 - ☐ [Checkout](#)
 - ☐ [Currency](#)
 - ☐ [Email](#)
 - ☐ [Flagd-UI](#)
 - ☐ [Fraud Detection](#)
 - ☐ [Frontend](#)
 - ☐ [Frontend Proxy](#)
 - ☐ [Image Provider](#)
 - ☐ [Kafka](#)
 - ☐ [Load Generator](#)
 - ☐ [Payment](#)
 - ☐ [Product Catalog](#)
 - ☐ [Product Reviews](#)
 - ☐ [Quote](#)
 - ☐ [React Native App](#)
 - ☐ [Recommendation](#)
 - ☐ [Shipping](#)
 - ☐ [Telemetry Features](#)
 - ☐ [Log Coverage](#)
 - ☐ [Manual Span Attributes](#)
 - ☐ [Metric Coverage](#)
 - ☐ [Trace Coverage](#)
 - ☐ [Tests](#)
 - ☐ [Language APIs & SDKs](#)

- ☐ [SDK Config](#)
 - ☐ [General](#)
 - ☐ [OTLP Exporter](#)
 - ☐ [Declarative configuration](#)
- ☐ [C++](#)
 - ☐ [Getting Started by Example](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [.NET](#)
 - ☐ [Getting Started](#)
 - ☐ [Traces](#)
 - ☐ [Console](#)
 - ☐ [ASP.NET Core](#)
 - ☐ [Stratified sampling](#)
 - ☐ [Tail-based sampling](#)
 - ☐ [Export to Jaeger](#)
 - ☐ [Exceptions](#)
 - ☐ [Links](#)
 - ☐ [Links-based sampling](#)
 - ☐ [Best practices](#)
 - ☐ [Metrics](#)
 - ☐ [Console](#)
 - ☐ [ASP.NET Core](#)
 - ☐ [Export to Prometheus](#)
 - ☐ [Exemplars](#)
 - ☐ [Instruments](#)
 - ☐ [Best practices](#)
 - ☐ [Logs](#)
 - ☐ [Console](#)
 - ☐ [ASP.NET Core](#)
 - ☐ [Complex objects](#)
 - ☐ [Correlation](#)
 - ☐ [Dedicated pipeline](#)
 - ☐ [Routing](#)
 - ☐ [Redaction](#)
 - ☐ [Best practices](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Resources](#)
 - ☐ [Exporters](#)
 - ☐ [Sampling](#)
 - ☐ [.NET Framework](#)
 - ☐ [Troubleshooting](#)
 - ☐ [Tracing Shim](#)
 - ☐ [API - tracing](#)
 - ☐ [API - metrics](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [Erlang/Elixir](#)
 - ☐ [Getting Started](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [Propagation](#)
 - ☐ [Resources](#)
 - ☐ [Sampling](#)
 - ☐ [Testing](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [Go](#)
 - ☐ [Getting Started](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [Resources](#)
 - ☐ [Sampling](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)

- ☐ [Java](#)
 - ☐ [Intro to OpenTelemetry Java](#)
 - ☐ [Getting Started by Example](#)
 - ☐ [Instrumentation ecosystem](#)
 - ☐ [Record Telemetry with API](#)
 - ☐ [Manage Telemetry with SDK](#)
 - ☐ [Configure the SDK](#)
 - ☐ [JMX Metrics](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [JavaScript](#)
 - ☐ [Getting Started by Example](#)
 - ☐ [Node.js](#)
 - ☐ [Browser](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [Context](#)
 - ☐ [Propagation](#)
 - ☐ [Resources](#)
 - ☐ [Sampling](#)
 - ☐ [Serverless](#)
 - ☐ [Benchmarks](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [Kotlin](#)
 - ☐ [Getting Started](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [PHP](#)
 - ☐ [Getting Started](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [Context](#)
 - ☐ [Propagation](#)
 - ☐ [Resources](#)
 - ☐ [SDK](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [Python](#)
 - ☐ [Getting Started by Example](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [Propagation](#)
 - ☐ [Cookbook](#)
 - ☐ [Distro](#)
 - ☐ [Using mypy](#)
 - ☐ [Benchmarks](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [Ruby](#)
 - ☐ [Getting Started](#)
 - ☐ [Instrumentation](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [Sampling](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [Rust](#)
 - ☐ [Getting Started](#)
 - ☐ [Libraries](#)
 - ☐ [Exporters](#)
 - ☐ [API](#)
 - ☐ [Examples](#)
 - ☐ [Registry](#)
- ☐ [Swift](#)
 - ☐ [Getting Started](#)
 - ☐ [Instrumentation](#)

- ☐ [Libraries](#)
- ☐ [Examples](#)
- ☐ [Registry](#)
- ☐ [Other](#)
- ☐ [Platforms](#)
 - ☐ [Client-side Apps](#)
 - ☐ [Android](#)
 - ☐ [iOS](#)
 - ☐ [Web](#)
 - ☐ [FaaS](#)
 - ☐ [Lambda Auto-Instrumentation](#)
 - ☐ [Lambda Collector Config](#)
 - ☐ [Lambda Manual Instrumentation](#)
 - ☐ [Kubernetes](#)
 - ☐ [Getting Started](#)
 - ☐ [Collector](#)
 - ☐ [Components](#)
 - ☐ [Helm Charts](#)
 - ☐ [Collector Chart](#)
 - ☐ [Demo Chart](#)
 - ☐ [Operator Chart](#)
 - ☐ [Kubernetes Operator](#)
 - ☐ [Auto-instrumentation](#)
 - ☐ [Horizontal Pod Autoscaling](#)
 - ☐ [Target Allocator](#)
 - ☐ [Troubleshooting](#)
 - ☐ [Auto-instrumentation](#)
 - ☐ [Prometheus Alerts Runbooks](#)
 - ☐ [Target Allocator](#)
- ☐ [Zero-code Instrumentation](#)
 - ☐ [OBI](#)
 - ☐ [Configure](#)
 - ☐ [Export modes](#)
 - ☐ [Global properties](#)
 - ☐ [Export data](#)
 - ☐ [Service discovery](#)
 - ☐ [Metrics attributes](#)
 - ☐ [Filter data](#)
 - ☐ [Routes decorator](#)
 - ☐ [Metrics histograms](#)
 - ☐ [Sample traces](#)
 - ☐ [Collector receiver](#)
 - ☐ [Internal metrics reporter](#)
 - ☐ [Tune performance](#)
 - ☐ [YAML example](#)
 - ☐ [Network](#)
 - ☐ [Measure traffic between Cloud availability zones](#)
 - ☐ [Quickstart](#)
 - ☐ [Configuration](#)
 - ☐ [Setup](#)
 - ☐ [Helm chart](#)
 - ☐ [Docker](#)
 - ☐ [Kubernetes](#)
 - ☐ [Standalone](#)
 - ☐ [Exported metrics](#)
 - ☐ [Distributed traces](#)
 - ☐ [Request times](#)
 - ☐ [Security](#)
 - ☐ [Troubleshooting](#)
 - ☐ [Cilium compatibility](#)
 - ☐ [Metrics cardinality](#)
 - ☐ [Trace-log correlation](#)
 - ☐ [Go](#)
 - ☐ [Auto SDK](#)
 - ☐ [.NET](#)
 - ☐ [Getting Started](#)
 - ☐ [Instrumentations](#)
 - ☐ [Configuration](#)
 - ☐ [Custom instrumentation](#)
 - ☐ [NuGet Packages](#)
 - ☐ [Troubleshooting](#)
 - ☐ [PHP](#)
 - ☐ [Python](#)
 - ☐ [Configuration](#)

- ☐ [Example](#)
- ☐ [Logs Example](#)
- ☐ [Operator](#)
- ☐ [Troubleshooting](#)
- ☐ [Java](#)
 - ☐ [Agent](#)
 - ☐ [Getting started](#)
 - ☐ [Configuration](#)
 - ☐ [Declarative configuration](#)
 - ☐ [Supported Libraries](#)
 - ☐ [Suppressing instrumentation](#)
 - ☐ [Annotations](#)
 - ☐ [Extend with the API](#)
 - ☐ [Instrumentation config](#)
 - ☐ [HTTP](#)
 - ☐ [App server config](#)
 - ☐ [Extensions](#)
 - ☐ [Performance](#)
 - ☐ [Quarkus](#)
 - ☐ [Spring Boot starter](#)
 - ☐ [Getting started](#)
 - ☐ [Extend with the API](#)
 - ☐ [Declarative configuration](#)
 - ☐ [SDK configuration](#)
 - ☐ [Programmatic configuration](#)
 - ☐ [Out of the box instrumentation](#)
 - ☐ [Annotations](#)
 - ☐ [Additional instrumentation](#)
 - ☐ [Other Spring autoconfiguration](#)
- ☐ [JavaScript](#)
 - ☐ [Configuration](#)
- ☐ [Collector](#)
 - ☐ [Quick start](#)
 - ☐ [Install](#)
 - ☐ [Docker](#)
 - ☐ [Kubernetes](#)
 - ☐ [Binary](#)
 - ☐ [Linux](#)
 - ☐ [macOS](#)
 - ☐ [Windows](#)
 - ☐ [Deploy](#)
 - ☐ [Agent pattern](#)
 - ☐ [Gateway pattern](#)
 - ☐ [Other patterns](#)
 - ☐ [Agent-to-gateway pattern](#)
 - ☐ [No Collector](#)
 - ☐ [Configuration](#)
 - ☐ [Components](#)
 - ☐ [Receivers](#)
 - ☐ [Processors](#)
 - ☐ [Exporters](#)
 - ☐ [Connectors](#)
 - ☐ [Extensions](#)
 - ☐ [Management](#)
 - ☐ [Distributions](#)
 - ☐ [Internal telemetry](#)
 - ☐ [Troubleshooting](#)
 - ☐ [Scaling the Collector](#)
 - ☐ [Transforming telemetry](#)
 - ☐ [Architecture](#)
 - ☐ [Extend](#)
 - ☐ [Build from source](#)
 - ☐ [Build a custom Collector](#)
 - ☐ [Build custom components](#)
 - ☐ [Receivers](#)
 - ☐ [Connectors](#)
 - ☐ [Extensions](#)
 - ☐ [Authenticator](#)
 - ☐ [Benchmarks](#)
 - ☐ [Registry](#)
 - ☐ [Resiliency](#)
- ☐ [Blueprints and reference implementations](#)
 - ☐ [Blueprints](#)
 - ☐ [Infrastructure and Processes in Non-K8s Environments](#)

- ☐ [Reference implementations](#)
 - ☐ [Adobe](#)
 - ☐ [Mastodon](#)
 - ☐ [Skyscanner](#)
- ☐ [Compatibility](#)
 - ☐ [Prometheus](#)
 - ☐ [Client libraries](#)
 - ☐ [OTLP metrics export](#)
 - ☐ [Migration](#)
 - ☐ [OpenCensus](#)
 - ☐ [OpenTracing](#)
- ☐ [Specs](#)
 - ☐ [Status](#)
 - ☐ [Configuration Types](#)
 - ☐ [OTel 1.57.0](#)
 - ☐ [Overview](#)
 - ☐ [Baggage](#)
 - ☐ [API](#)
 - ☐ [Client Design Principles](#)
 - ☐ [Common concepts](#)
 - ☐ [Attribute Naming](#)
 - ☐ [Attribute Requirement Levels](#)
 - ☐ [Instrumentation Scope](#)
 - ☐ [Mapping to AnyValue](#)
 - ☐ [Mapping to non-OTLP Formats](#)
 - ☐ [Compatibility](#)
 - ☐ [OpenCensus](#)
 - ☐ [OpenTracing](#)
 - ☐ [Prometheus and OpenMetrics](#)
 - ☐ [Trace Context in non-OTLP Log Formats](#)
 - ☐ [Configuration](#)
 - ☐ [API](#)
 - ☐ [Data Model](#)
 - ☐ [SDK](#)
 - ☐ [Configuration Types](#)
 - ☐ [Common](#)
 - ☐ [Env var](#)
 - ☐ [Supplementary Guidelines](#)
 - ☐ [Context](#)
 - ☐ [Environment Variables as Context Propagation Carriers](#)
 - ☐ [Propagators API](#)
 - ☐ [Definitions of Document Statuses](#)
 - ☐ [Entities](#)
 - ☐ [Data Model](#)
 - ☐ [Entity Propagation](#)
 - ☐ [Error handling in OpenTelemetry](#)
 - ☐ [Glossary](#)
 - ☐ [Logs](#)
 - ☐ [API](#)
 - ☐ [Data Model](#)
 - ☐ [SDK](#)
 - ☐ [Data Model Appendix](#)
 - ☐ [Exporters](#)
 - ☐ [Stdout](#)
 - ☐ [No-Op](#)
 - ☐ [Supplementary Guidelines](#)
 - ☐ [Metrics](#)
 - ☐ [API](#)
 - ☐ [Data Model](#)
 - ☐ [SDK](#)
 - ☐ [Exporters](#)
 - ☐ [In-memory](#)
 - ☐ [OTLP](#)
 - ☐ [Prometheus](#)
 - ☐ [Stdout](#)
 - ☐ [Metric Requirement Levels](#)
 - ☐ [No-Op](#)
 - ☐ [Supplementary Guidelines](#)
 - ☐ [Performance and Blocking of OpenTelemetry API](#)
 - ☐ [Performance Benchmark of OpenTelemetry API](#)
 - ☐ [Profiles](#)
 - ☐ [Mappings](#)
 - ☐ [Pprof](#)
 - ☐ [Project Package Layout](#)

- ☐ [Protocol](#)
 - ☐ [Specification 1.10.0](#)
 - ☐ [Design Goals](#)
 - ☐ [Exporter](#)
 - ☐ [File Exporter](#)
 - ☐ [Requirements](#)
- ☐ [Resource](#)
 - ☐ [Data Model](#)
 - ☐ [SDK](#)
- ☐ [Schemas](#)
 - ☐ [1.0.0](#)
 - ☐ [1.1.0](#)
- ☐ [Semantic Conventions](#)
- ☐ [Specification Principles](#)
- ☐ [Telemetry Stability](#)
- ☐ [The OpenTelemetry approach to upgrading](#)
- ☐ [Trace](#)
 - ☐ [API](#)
 - ☐ [SDK](#)
 - ☐ [Exceptions](#)
 - ☐ [Exporters](#)
 - ☐ [Stdout](#)
 - ☐ [Zipkin](#)
 - ☐ [Probability Sampling](#)
 - ☐ [TraceState](#)
- ☐ [Vendors](#)
- ☐ [Versioning and stability for OpenTelemetry clients](#)
- ☐ [OTLP 1.10.0](#)
- ☐ [OpAMP](#)
- ☐ [Semantic conventions 1.41.1](#)
- ☐ [Registry](#)
 - ☐ [Attributes](#)
 - ☐ [Android](#)
 - ☐ [App](#)
 - ☐ [Artifact](#)
 - ☐ [Aspnetcore](#)
 - ☐ [AWS](#)
 - ☐ [Azure](#)
 - ☐ [Browser](#)
 - ☐ [Cassandra](#)
 - ☐ [CICD](#)
 - ☐ [Client](#)
 - ☐ [Cloud](#)
 - ☐ [CloudEvents](#)
 - ☐ [CloudFoundry](#)
 - ☐ [Code](#)
 - ☐ [Container](#)
 - ☐ [CPU](#)
 - ☐ [CPython](#)
 - ☐ [DB](#)
 - ☐ [Deployment](#)
 - ☐ [Destination](#)
 - ☐ [Device](#)
 - ☐ [Disk](#)
 - ☐ [DNS](#)
 - ☐ [Dotnet](#)
 - ☐ [Elasticsearch](#)
 - ☐ [Enduser](#)
 - ☐ [Error](#)
 - ☐ [Event](#)
 - ☐ [Exception](#)
 - ☐ [Faas](#)
 - ☐ [Feature flag](#)
 - ☐ [File](#)
 - ☐ [GCP](#)
 - ☐ [Gen AI](#)
 - ☐ [Geo](#)
 - ☐ [Go](#)
 - ☐ [GraphQL](#)
 - ☐ [Hardware](#)
 - ☐ [Heroku](#)
 - ☐ [Host](#)
 - ☐ [HTTP](#)
 - ☐ [iOS](#)

- ☐ [JSONRPC](#)
- ☐ [JVM](#)
- ☐ [K8s](#)
- ☐ [Linux](#)
- ☐ [Log](#)
- ☐ [Mainframe](#)
- ☐ [MCP](#)
- ☐ [Messaging](#)
- ☐ [Network](#)
- ☐ [NFS](#)
- ☐ [NodeJS](#)
- ☐ [OCI](#)
- ☐ [ONC RPC](#)
- ☐ [OpenAI](#)
- ☐ [Openshift](#)
- ☐ [OpenTracing](#)
- ☐ [Oracle cloud](#)
- ☐ [OracleDB](#)
- ☐ [OS](#)
- ☐ [OTel](#)
- ☐ [Peer](#)
- ☐ [Pprof](#)
- ☐ [Process](#)
- ☐ [Profile](#)
- ☐ [RPC](#)
- ☐ [Security rule](#)
- ☐ [Server](#)
- ☐ [Service](#)
- ☐ [Session](#)
- ☐ [SignalR](#)
- ☐ [Source](#)
- ☐ [System](#)
- ☐ [Telemetry](#)
- ☐ [Test](#)
- ☐ [Thread](#)
- ☐ [TLS](#)
- ☐ [URL](#)
- ☐ [User](#)
- ☐ [User agent](#)
- ☐ [V8js](#)
- ☐ [VCS](#)
- ☐ [Webengine](#)
- ☐ [zOS](#)
- ☐ [Entities](#)
 - ☐ [Android](#)
 - ☐ [App](#)
 - ☐ [AWS](#)
 - ☐ [Browser](#)
 - ☐ [CICD](#)
 - ☐ [Cloud](#)
 - ☐ [CloudFoundry](#)
 - ☐ [Container](#)
 - ☐ [Deployment](#)
 - ☐ [Device](#)
 - ☐ [Faas](#)
 - ☐ [GCP](#)
 - ☐ [Heroku](#)
 - ☐ [Host](#)
 - ☐ [K8s](#)
 - ☐ [Openshift](#)
 - ☐ [OS](#)
 - ☐ [OTel](#)
 - ☐ [Process](#)
 - ☐ [Service](#)
 - ☐ [Telemetry](#)
 - ☐ [VCS](#)
 - ☐ [Webengine](#)
 - ☐ [zOS](#)
- ☐ [General](#)
 - ☐ [Attribute requirement levels](#)
 - ☐ [Attributes](#)
 - ☐ [Events](#)
 - ☐ [Logs](#)
 - ☐ [Metric requirement levels](#)

- ☐ [Metrics](#)
- ☐ [Naming](#)
- ☐ [Profiles](#)
- ☐ [Recording errors](#)
- ☐ [Semantic convention groups](#)
- ☐ [Session](#)
- ☐ [Trace](#)
- ☐ [Tracing compatibility](#)
- ☐ [.NET](#)
 - ☐ [ASP.NET Core](#)
 - ☐ [DNS](#)
 - ☐ [HTTP](#)
 - ☐ [HTTP request and connection spans](#)
 - ☐ [Kestrel](#)
 - ☐ [SignalR](#)
- ☐ [App](#)
 - ☐ [Events](#)
- ☐ [Azure](#)
 - ☐ [Events](#)
- ☐ [Browser](#)
 - ☐ [Events](#)
- ☐ [CICD](#)
 - ☐ [Logs](#)
 - ☐ [Metrics](#)
 - ☐ [Spans](#)
- ☐ [CLI programs](#)
 - ☐ [Spans](#)
- ☐ [Cloud providers](#)
 - ☐ [AWS SDK](#)
- ☐ [CloudEvents](#)
 - ☐ [Spans](#)
- ☐ [Configuration](#)
 - ☐ [Version selection](#)
- ☐ [Database](#)
 - ☐ [Cassandra](#)
 - ☐ [Cosmos DB](#)
 - ☐ [CouchDB](#)
 - ☐ [DynamoDB](#)
 - ☐ [Elasticsearch](#)
 - ☐ [Exceptions](#)
 - ☐ [HBase](#)
 - ☐ [MariaDB](#)
 - ☐ [Metrics](#)
 - ☐ [MongoDB](#)
 - ☐ [MySQL](#)
 - ☐ [Oracle Database](#)
 - ☐ [PostgreSQL](#)
 - ☐ [Redis](#)
 - ☐ [Spans](#)
 - ☐ [SQL](#)
 - ☐ [SQL Server](#)
- ☐ [DNS](#)
 - ☐ [Metrics](#)
- ☐ [Exceptions](#)
 - ☐ [Logs](#)
 - ☐ [Spans](#)
- ☐ [FaaS](#)
 - ☐ [AWS Lambda](#)
 - ☐ [Exceptions](#)
 - ☐ [Metrics](#)
 - ☐ [Spans](#)
- ☐ [Feature flags](#)
 - ☐ [Events](#)
- ☐ [Generative AI](#)
 - ☐ [Agent spans](#)
 - ☐ [Anthropic](#)
 - ☐ [AWS Bedrock](#)
 - ☐ [Azure AI Inference](#)
 - ☐ [Events](#)
 - ☐ [Exceptions](#)
 - ☐ [LLM call examples](#)
 - ☐ [Metrics](#)
 - ☐ [Model Context Protocol](#)
 - ☐ [OpenAI](#)

- ☐ [Spans](#)
- ☐ [GraphQL](#)
 - ☐ [GraphQL server](#)
- ☐ [Hardware](#)
 - ☐ [Battery](#)
 - ☐ [CPU](#)
 - ☐ [Disk Controller](#)
 - ☐ [Enclosure](#)
 - ☐ [Fan](#)
 - ☐ [GPU](#)
 - ☐ [Logical Disk](#)
 - ☐ [Memory](#)
 - ☐ [Metrics](#)
 - ☐ [Network](#)
 - ☐ [Physical Disk](#)
 - ☐ [Physical host](#)
 - ☐ [Power Supply](#)
 - ☐ [Tape Drive](#)
 - ☐ [Temperature](#)
 - ☐ [Voltage](#)
- ☐ [How to write conventions](#)
 - ☐ [Resource and Entities](#)
 - ☐ [Status Metrics](#)
 - ☐ [T-shaped Signals](#)
- ☐ [HTTP](#)
 - ☐ [Exceptions](#)
 - ☐ [Metrics](#)
 - ☐ [Spans](#)
- ☐ [Messaging](#)
 - ☐ [AWS SNS](#)
 - ☐ [AWS SQS](#)
 - ☐ [Azure](#)
 - ☐ [Exceptions](#)
 - ☐ [Google Cloud Pub/Sub](#)
 - ☐ [Kafka](#)
 - ☐ [Metrics](#)
 - ☐ [RabbitMQ](#)
 - ☐ [RocketMQ](#)
 - ☐ [Spans](#)
- ☐ [Mobile](#)
 - ☐ [Events](#)
- ☐ [NFS](#)
 - ☐ [NFS](#)
- ☐ [Non-normative](#)
 - ☐ [Code attributes migration](#)
 - ☐ [Compatibility](#)
 - ☐ [AWS](#)
 - ☐ [gRPC](#)
 - ☐ [Database migration](#)
 - ☐ [Generating semantic convention libraries](#)
 - ☐ [HTTP migration](#)
 - ☐ [K8s attributes](#)
 - ☐ [K8s migration](#)
 - ☐ [Naming known exceptions](#)
 - ☐ [Recommended vs Opt-In CPU Metrics](#)
 - ☐ [RPC migration](#)
 - ☐ [System semantic conventions: instrumentation design philosophy](#)
 - ☐ [System use cases](#)
- ☐ [Object stores](#)
 - ☐ [S3](#)
- ☐ [OpenTelemetry SDK](#)
 - ☐ [SDK Metrics](#)
- ☐ [Resource](#)
 - ☐ [Android](#)
 - ☐ [Browser](#)
 - ☐ [CICD](#)
 - ☐ [Cloud](#)
 - ☐ [Cloud provider](#)
 - ☐ [AWS](#)
 - ☐ [ECS](#)
 - ☐ [EKS](#)
 - ☐ [Logs](#)
 - ☐ [GCP](#)
 - ☐ [Google Cloud AppHub](#)

- ☐ [Google Cloud Run](#)
- ☐ [Google Compute Engine](#)
- ☐ [Heroku](#)
- ☐ [CloudFoundry](#)
- ☐ [Container](#)
- ☐ [Deployment](#)
- ☐ [Device](#)
- ☐ [FaaS](#)
- ☐ [Host](#)
- ☐ [Kubernetes](#)
 - ☐ [Openshift](#)
- ☐ [Operating system](#)
- ☐ [Process](#)
- ☐ [Service](#)
- ☐ [Webengine](#)
- ☐ [z/OS software](#)
- ☐ [RPC](#)
 - ☐ [Connect](#)
 - ☐ [Dubbo](#)
 - ☐ [Exceptions](#)
 - ☐ [gRPC](#)
 - ☐ [JSON-RPC](#)
 - ☐ [Metrics](#)
 - ☐ [Spans](#)
- ☐ [Runtime environment](#)
 - ☐ [.NET](#)
 - ☐ [CPython](#)
 - ☐ [Go](#)
 - ☐ [JVM](#)
 - ☐ [Node.js](#)
 - ☐ [V8 JS engine](#)
- ☐ [System](#)
 - ☐ [Container](#)
 - ☐ [Kubernetes](#)
 - ☐ [OpenShift](#)
 - ☐ [OS process](#)
 - ☐ [System](#)
- ☐ [URL](#)
- ☐ [Security](#)
 - ☐ [Common Vulnerabilities and Exposures](#)
 - ☐ [Handling sensitive data](#)
 - ☐ [Community incident response guidelines](#)
 - ☐ [Collector configuration](#)
 - ☐ [Collector hosting](#)
- ☐ [Contributing](#)
 - ☐ [Prerequisites](#)
 - ☐ [Issues](#)
 - ☐ [Submitting content](#)
 - ☐ [Style guide](#)
 - ☐ [Localization](#)
 - ☐ [Blog](#)
 - ☐ [PR checks & tests](#)
 - ☐ [Announcements](#)
 - ☐ [Dev setup and more](#)
 - ☐ [SIG practices](#)
 - ☐ [Acknowledgements](#)

[View Markdown](#) [View page source](#) [Edit this page](#) [Create child page](#) [Create documentation issue](#)

On this page

- [Terminology](#)
- [Why sampling?](#)
 - [When to sample](#)
 - [When not to sample](#)
- [Head Sampling](#)
- [Tail Sampling](#)
- [Support](#)
 - [Collector](#)
 - [Language SDKs](#)
 - [Vendors](#)

1. [Docs](#)
2. [Concepts](#)
3. Sampling

Sampling

Learn about sampling and the different sampling options available in OpenTelemetry.

With [traces](#), you can observe requests as they move from one service to another in a distributed system. Tracing is highly practical for both high-level and in-depth analysis of systems.

However, if the large majority of your requests are successful and finish with acceptable latency and no errors, you do not need 100% of your traces to meaningfully observe your applications and systems. You just need the right sampling.



Terminology

It's important to use consistent terminology when discussing sampling. A trace or span is considered “sampled” or “not sampled”:

- **Sampled:** A trace or span is processed and exported. Because it is chosen by the sampler as a representative of the population, it is considered “sampled”.
- **Not sampled:** A trace or span is not processed or exported. Because it is not chosen by the sampler, it is considered “not sampled”.

Sometimes, the definitions of these terms get mixed up. You might find someone states that they are “sampling out data” or that data not processed or exported is considered “sampled”. These are incorrect statements.

Why sampling?

Sampling is one of the most effective ways to reduce the costs of observability without losing visibility. Although there are other ways to lower costs, such as filtering or aggregating data, these other methods do not adhere to the concept of representativeness, which is crucial when performing in-depth analysis of application or system behavior.

Representativeness is the principle that a smaller group can accurately represent a larger group. Additionally, representativeness can be mathematically verified, meaning that you can have high confidence that a smaller sample of data accurately represents the larger group.

Additionally, the more data you generate, the less data you actually need to have a representative sample. For high-volume systems, it is quite common for a sampling rate of 1% or lower to very accurately represent the other 99% of data.

When to sample

Consider sampling if you meet any of the following criteria:

- You generate 1000 or more traces per second.
- Most of your trace data represents healthy traffic with little variation in data.
- You have some common criteria, like errors or high latency, that usually means something is wrong.
- You have domain-specific criteria you can use to determine relevant data beyond errors and latency.
- You can describe some common rules that determine if data should be sampled or dropped.
- You have a way to tell your services apart, so that high- and low-volume services are sampled differently.
- You have the ability to route unsampled data (for “just in case” scenarios) to low-cost storage systems.

Finally, consider your overall budget. If you have limited budget for observability, but can afford to spend time to effectively sample, then sampling can generally be worth it.

When not to sample

Sampling might not be appropriate for you. You might want to avoid sampling if you meet any of the following criteria:

- You generate very little data (tens of small traces per second or lower).
- You only use observability data in aggregate, and can thus pre-aggregate data.
- You are bound by circumstances such as regulation that prohibit dropping data (and cannot route unsampled data to low-cost storage).

Finally, consider the following three costs associated with sampling:

1. The direct cost of compute to effectively sample data, such as a tail sampling proxy.
2. The indirect engineering cost of maintaining effective sampling methodologies as more applications, systems, and data are involved.
3. The indirect opportunity cost of missing critical information with ineffective sampling techniques.

Sampling, while effective at reducing observability costs, might introduce other unexpected costs if not performed well. It could be cheaper to allocate more resources for observability instead, either with a vendor or compute when self-hosting, depending on your observability backend, the nature of your data, and your attempts to sample effectively.

Head Sampling

Head sampling is a sampling technique used to make a sampling decision as early as possible. A decision to sample or drop a

span or trace is not made by inspecting the trace as a whole.

For example, the most common form of head sampling is [Consistent Probability Sampling](#). This is also referred to as Deterministic Sampling. In this case, a sampling decision is made based on the trace ID and the desired percentage of traces to sample. This ensures that whole traces are sampled - no missing spans - at a consistent rate, such as 5% of all traces.

The upsides to head sampling are:

- Easy to understand
- Easy to configure
- Efficient
- Can be done at any point in the trace collection pipeline

The primary downside to head sampling is that it is not possible to make a sampling decision based on data in the entire trace. For example, you cannot ensure that all traces with an error within them are sampled with head sampling alone. For this situation and many others, you need tail sampling.

Tail Sampling

Tail sampling is where the decision to sample a trace takes place by considering all or most of the spans within the trace. Tail Sampling gives you the option to sample your traces based on specific criteria derived from different parts of a trace, which isn't an option with Head Sampling.



Some examples of how you can use Tail Sampling include:

- Always sampling traces that contain an error
- Sampling traces based on overall latency
- Sampling traces based on the presence or value of specific attributes on one or more spans in a trace; for example, sampling more traces originating from a newly deployed service
- Applying different sampling rates to traces based on certain criteria, such as when traces only come from low-volume services versus traces with high-volume services.

As you can see, tail sampling allows for a much higher degree of sophistication in how you sample data. For larger systems that must sample telemetry, it is almost always necessary to use Tail Sampling to balance data volume with the usefulness of that data.

There are three primary downsides to tail sampling today:

1. Tail sampling can be difficult to implement. Depending on the kind of sampling techniques available to you, it is not always a “set and forget” kind of thing. As your systems change, so too will your sampling strategies. For a large and sophisticated distributed system, rules that implement sampling strategies can also be large and sophisticated.
2. Tail sampling can be difficult to operate. The component(s) that implement tail sampling must be stateful systems that can accept and store a large amount of data. Depending on traffic patterns, this can require dozens or even hundreds of compute nodes that all utilize resources differently. Furthermore, a tail sampler might need to “fall back” to less computationally intensive sampling techniques if it is unable to keep up with the volume of data it is receiving. Because of these factors, it is critical to monitor tail-sampling components to ensure that they have the resources they need to make the correct sampling decisions.
3. Tail samplers often end up as vendor-specific technology today. If you’re using a paid vendor for Observability, the most effective tail sampling options available to you might be limited to what the vendor offers.

Finally, for some systems, tail sampling might be used in conjunction with Head Sampling. For example, a set of services that produce an extremely high volume of trace data might first use head sampling to sample only a small percentage of traces, and then later in the telemetry pipeline use tail sampling to make more sophisticated sampling decisions before exporting to a backend. This is often done in the interest of protecting the telemetry pipeline from being overloaded.

Support

Collector

The OpenTelemetry Collector includes the following sampling processors:

- [Probabilistic Sampling Processor](#)
- [Tail Sampling Processor](#)

Language SDKs

For the individual language-specific implementations of the OpenTelemetry API & SDK, you will find support for sampling in the respective documentation pages:

- [.NET](#)
- [Erlang/Elixir](#)
- [Go](#)
- [JavaScript](#)
- [Ruby](#)

Vendors

Many [vendors](#) offer comprehensive sampling solutions that incorporate head sampling, tail sampling, and other features that can support sophisticated sampling needs. These solutions may also be optimized specifically for the vendor’s backend. If you are sending telemetry to a vendor, consider using their sampling solutions.

Feedback

Was this page helpful?

Yes

No

Last modified October 16, 2025: [Replace OTEP 4673 links by reference to OTel spec trace/tracestate-probability-sampling \(#8108\) \(6b2e90b3\)](#)

-
-
-
-
-
-
-
-
-
-
-
-
-
-
-